Bidirectional Wireless Bridge: S7-200 PLC to ESP32

This document details the architecture for a bidirectional communication system. The **Gateway ESP32** acts as a "Translator," moving data from a wireless sensor network to a wired industrial S7-200 PLC, and vice versa.

1. System Components

A. The PLC Station (Local)

1. **S7-200 PLC:** The legacy industrial "Master" (e.g., CPU 222, 224, or 226) that controls the logic using Step 7-Micro/WIN.
2. **Physical PLC Button:** Wired directly to a Digital Input (e.g., $I0.0$). This is a local sensor.
3. **Physical PLC Bulb:** Wired directly to a Digital Output (e.g., $Q0.0$). This is a local actuator.
4. **MAX485 Module:** Translates the PLC's RS-485 signals (from Port 0 or Port 1) into TTL for the Gateway.

B. The Gateway (The Bridge)

1. **Gateway Node (ESP32):** The central hub.
2. **Reset Button:** A physical button on the ESP32 used to clear the "Memory Map" (virtual registers) to ensure system sync.

C. The Remote Node (Wireless)

1. **Remote Wireless Node (ESP32-S3):** The remote prototype station.
2. **Remote Push Buttons:** These act as wireless sensors (Inputs).
3. **Onboard/External LEDs:** These act as wireless actuators (Outputs) for the prototype samples.

2. The Gateway "Memory Map"

The Gateway maintains a table of registers. This is the "Shared Clipboard" for the system.

Register Address	Name	Data Flow
Register 40001	Remote Button State	**Wireless** $\rightarrow$ **Gateway** $\rightarrow$ **PLC**
Register 40002	Remote LED Command	**PLC** $\rightarrow$ **Gateway** $\rightarrow$

		Wireless

3. Interaction Logic: "The Mirror Effect"

In this setup, the PLC logic links the physical world with the wireless world.

Scenario A: Remote Wireless Button controls Local PLC Bulb

1. A user presses the **Push Button** on the **Remote ESP32-S3**.
2. The state (1) is sent via **ESP-NOW** to the Gateway.
3. The Gateway saves 1 in **Register 40001**.
4. The **S7-200 PLC** reads Register 40001 via **Modbus RTU** (using MBUS_MSG).
5. **PLC Logic:** "If Register 40001 is 1, then turn on **Physical PLC Bulb (Q0.0)**."

Scenario B: Local PLC Button controls Remote Wireless LEDs

1. **PLC Logic:** "If **Physical Button (I0.0)** is 1, then write 1 to **Register 40002**."
2. The PLC sends a Modbus command to the Gateway to update Register 40002.
3. The **Gateway** detects this change and sends an **ESP-NOW** signal to the remote node.
4. The **Remote ESP32-S3** receives the signal and turns on the **LEDs**.

4. The Gateway Reset Button

The button on the Gateway is dedicated to **Safety and Sync**.

- If the communication is interrupted, pressing the **Gateway Reset** sets all registers in the Memory Map to 0.
- This forces both the PLC and the wireless node to "handshake" again and update their true states.

5. Communication Summary

- **Wired Link:** S7-200 (Port 0/1) $\leftrightarrow$ MAX485 $\leftrightarrow$ Gateway ESP32 (Modbus RTU).
- **Wireless Link:** Gateway ESP32 $\leftrightarrow$ Remote ESP32-S3 (ESP-NOW).
- **Logic Master:** The **S7-200 PLC** makes all decisions.

6. The "Message Forms" (Data Structure)

A. The Wired Form (From PLC via MAX485)

The PLC uses standard Modbus RTU frames.

Example Frame: [01] [06] [00 01] [00 01] [D8 0A] (Note: Modbus address 40002 is often 0001 in hex offset).

B. The Wireless Form (ESP-NOW Payload)

The "Message" as Text: CMD:LED_1:1.

7. Storage: Where are the Registers?

1. **Gateway SRAM:** The ESP32 holds the values in volatile memory.
2. **PLC V-Memory:** The S7-200 stores the results in its **V-Memory** (e.g., VW100), which is mapped during the Modbus transfer.

8. The Communication Cycle: Polling

1. **S7-200** is the **Master**.
2. It uses the MBUS_CTRL block to initialize the port.
3. It uses the MBUS_MSG block to "Read" or "Write" to the ESP32 Gateway.